Data Structures and algorithms
Instructor : **Dr.Amin Eskandari**
Head-Ta's : Amir Mohammad Asadjoo, Ali Nikouan
Ta's: Mehdi Heidari, Narges Hosseini, Abolfazl Tadrisi, Arvin Afshari, Amir Arshia Tootoonchi, Fatemeh Feyzi

پاسخ سوال ۱ (۱۰ امتیاز)

راه‌حل

نکته‌ی کلیدی در صورت سوال این است که نمره یک عدد صحیح بین ۰ تا ۱۰۰ است. یعنی دو پیش‌شرط استفاده از مرتب‌سازی شمارشی (Counting Sort)برقرار است: اعداد صحیح و نامنفی‌اند، و بزرگ‌ترین مقدار ممکن را از قبل می‌دانیم و خیلی بزرگ نیست.
الگوریتم در دو گام کار می‌کند:
گام اول (شمارش): یک آرایه‌ی کمکی به نام Countبا اندازه‌ی m + 1می‌سازیم و همه‌ی خانه‌ها را صفر می‌گذاریم. سپس یک بار آرایه‌ی ورودی را پیمایش می‌کنیم و به ازای هر عدد x، مقدار Count[x]را یکی زیاد می‌کنیم. در پایان این گام، Count[i] برابر تعداد تکرار عدد iدر ورودی است.

گام دوم (بازسازی): از عدد ۰ تا m پیش می‌رویم و هر عدد iرا دقیقاً Count[i]بار در انتهای آرایه‌ی خروجی می‌گذاریم. چون از کوچک به بزرگ پیش رفته‌ایم، خروجی به‌طور خودکار مرتب است.

بعد از ساختن آرایه‌ی Count، پیدا کردن پرتکرارترین نمره تقریباً رایگان است: روی Count می‌گردیم و اندیسی را پیدا می‌کنیم که مقدارش بیشینه باشد. برای رعایت شرط «کوچک‌ترین نمره در حالت تساوی»، از شرط بزرگ‌تر اکید (>) استفاده می‌کنیم و از چپ به راست حرکت می‌کنیم؛ در این صورت وقتی به تساوی می‌رسیم، مقدار قبلی که اندیس کوچک‌تری دارد حفظ می‌شود.

چرا این راه‌حل و نه راه‌حل دیگر؟

استفاده از توابع آماده ممنوع است، پس دانشجو باید خودش یک الگوریتم مرتب‌سازی بنویسد. در این شرایط مرتب‌سازی شمارشی هم ساده‌ترین گزینه است و هم شرایطش کاملاً برقرار است، و در زمان O(n + m) تمام می‌شود که با m ثابت و برابر ۱۰۰، عملاً خطی نسبت به n است. با محدودیت‌های این سوال هر الگوریتم درست دیگری هم پذیرفته است، ولی هدف آموزشی سوال همین یکی است.

توجه کنید که این از کران پایین Omega(n log n) برای مرتب‌سازی‌های مقایسه‌ای بهتر است؛ چون این الگوریتم اصلاً هیچ دو عنصری را با هم مقایسه نمی‌کند و به همین دلیل آن کران پایین شاملش نمی‌شود.

اشتباهات رایج و پاسخ به پرسش‌های احتمالی دانشجو

- ساختن آرایه‌ی Count با اندازه‌ی m به‌جای m + 1 در این حالت به ازای ورودی حاوی نمره‌ی ۱۰۰ خطای IndexError می‌گیرید. اگر دانشجویی گفت «کدم روی بعضی تست‌ها ارور می‌دهد»، اول همین را چک کنید.
- استفاده از max(A) به‌جای مقدار ثابت ۱۰۰ برای اندازه‌ی Count این کار غلط نیست و هر دو رویکرد جواب درست می‌دهند، ولی اگر آرایه خالی از عناصر بزرگ باشد اندازه‌ی Count کوچکتر می‌شود که مشکلی ایجاد نمی‌کند.
- استفاده از شرط >= به‌جای > هنگام پیدا کردن بیشینه، که در حالت تساوی بزرگترین نمره را برمی‌گرداند نه کوچکترین.
- خواندن ورودی سطر به سطر با فرض اینکه همه‌ی نمره‌ها حتماً در یک سطر هستند. امن‌ترین کار خواندن کل ورودی و جدا کردن توکن‌هاست.

پیچیدگی

- زمان: O(n + m)
- حافظه: O(n + m)

پاسخ سوال ۲ (۳۰ امتیاز)

راه‌حل

فرایندی که صورت سوال توصیف کرده دقیقاً مرتب‌سازی درجی (Insertion Sort) است. در هر مرحله فرض می‌کنیم بخش A[0..k-1] از قبل مرتب است، عنصر A[k] را برمی‌داریم و آن را از سمت راست با عناصر بخش مرتب مقایسه می‌کنیم. هر عنصری که از آن اکیداً بزرگتر بود یک خانه به راست منتقل می‌شود تا جا باز شود، و به‌محض رسیدن به عنصری که بزرگتر نیست یا رسیدن به ابتدای آرایه متوقف می‌شویم.

نکته‌ی مهم پیاده‌سازی: در حلقه‌ی داخلی به‌جای swap کردن پیاپی، مقدار A[k] را یک بار در متغیر item ذخیره می‌کنیم و فقط انتقال انجام می‌دهیم. این کار هم سریع‌تر است (به‌جای سه انتساب برای هر swap، یک انتساب) و هم شمارش انتقال‌ها را دقیق و بدون ابهام می‌کند.

Data Structures and algorithms
Instructor : **Dr.Amin Eskandari**
Head-Ta's : Amir Mohammad Asadjoo, Ali Nikouan
Ta's: **Mehdi Heidari**, **Narges Hosseini**, **Abolfazl Tadrisi**, **Arvin Afshari**, **Amir Arshia Tootoonchi**,

تعداد انتقال‌ها چه عددی است؟

تعداد انتقال‌ها دقیقاً برابر تعداد نابه‌جایی‌های (inversion) آرایه‌ی اولیه است؛ یعنی تعداد زوج‌های (i, j)که در آن‌ها i < jولی A[i] > A[j]

دلیلش این است که هر انتقال دقیقاً یک نابه‌جایی را از بین می‌برد: وقتی item از روی یک عنصر بزرگتر رد می‌شود، آن زوج نابه‌جا اصلاح می‌شود و ترتیب نسبی هیچ زوج دیگری تغییر نمی‌کند. آرایه در پایان هیچ نابه‌جایی ندارد، پس تعداد کل انتقال‌ها برابر تعداد نابه‌جایی‌های آرایه‌ی اولیه است.

اگر دانشجویی پرسید «چطور جوابم را بدون اجرای کد چک کنم؟»، همین رابطه بهترین ابزار است: تعداد نابه‌جایی‌ها را با دو حلقه‌ی ساده بشمارد و با خروجی برنامه‌اش مقایسه کند.

تحلیل پیچیدگی زمانی

- بهترین حالت (آرایه‌ی از پیش مرتب): حلقه‌ی داخلی هیچ‌وقت اجرا نمی‌شود؛ هر عنصر فقط یک مقایسه با عنصر قبلی خود دارد. زمان اجرا O(n)و تعداد انتقال‌ها صفر است.
- بدترین حالت (آرایه‌ی معکوس مرتب): عنصر k-ام باید از روی هر kعنصر قبلی رد شود، پس تعداد کل انتقال‌ها برابر n(n-1)/2و زمان اجرا O(n^2)است.
- حالت میانگین: به‌طور متوسط نصف نابه‌جایی‌های ممکن وجود دارند، یعنی حدود n(n-1)/4 انتقال که باز هم O(n^2)است.

همین وابستگی شدید به ورودی است که این الگوریتم را تطبیقی (Adaptive)می‌کند و دلیل استفاده از آن در Timsort(الگوریتم پیش‌فرض پایتون و جاوا) برای آرایه‌های کوچک یا تقریباً مرتب است.

اشتباهات رایج و پاسخ به پرسش‌های احتمالی دانشجو

- چاپ کردن آرایه nبار به‌جای n-1بار. گزارش فقط برای k = 1 تا n-1لازم است، چون در k = 0هیچ درجی انجام نمی‌شود.
- شمردن مقایسه‌ها به‌جای انتقال‌ها. آخرین مقایسه‌ای که حلقه را متوقف می‌کند انتقالی به همراه ندارد و نباید شمرده شود. این شایع‌ترین دلیل اختلاف یک واحدی در هر مرحله است.

• استفاده از >= به‌جای > در شرط حلقه‌ی داخلی. با عناصر تکراری، این کار عدد انتقال‌ها را بیشتر از مقدار درست می‌کند.
• ثبت نکردن یک کپی از آرایه در گزارش (نوشتن trace.append(a) به‌جای trace.append(list(a))). چون لیست در پایتون با ارجاع منتقل می‌شود، در این صورت همه‌ی سطرهای گزارش یکسان و برابر آرایه‌ی نهایی می‌شوند. اگر دانشجو گفت «همه‌ی سطرهای خروجی‌ام مثل هم شده»، دقیقاً همین است.
• چاپ کردن سطر به سطر با print داخل حلقه در آرایه‌های بزرگ. جمع کردن خروجی در یک لیست و یک بار چاپ کردن آن سریع‌تر است.

پیچیدگی

• زمان: O(n^2) در بدترین حالت
• حافظه: O(n^2) برای نگهداری گزارش، که با n <= 1000 مشکلی ایجاد نمی‌کند

پاسخ سوال ۳ (۵۰ امتیاز)
راه‌حل

فرایند توصیف‌شده همان مرتب‌سازی حبابی (Bubble Sort) با بهینه‌سازی پرچم است، با این تفاوت که حلقه‌ی درونی کوتاه نمی‌شود و در هر دور کل آرایه پیمایش می‌گردد. هر سه خروجی را می‌توان با یک بار اجرای همین شبیه‌سازی به دست آورد؛ کافی است دو شمارنده نگه داریم.

نکته‌ی ظریف در شمارش دورها: شمارنده باید در ابتدای بدنه‌ی حلقه زیاد شود، نه بعد از بررسی پرچم. دوری که در آن هیچ جابه‌جایی رخ نمی‌دهد هم یک پیمایش کامل است و طبق صورت سوال باید شمرده شود. این شایع‌ترین دلیل اختلاف یک واحدی در سطر دوم خروجی است.

دو رابطه‌ی مفید برای تصحیح و پاسخ به دانشجو

رابطه‌ی اول: تعداد کل جابه‌جایی‌ها دقیقاً برابر تعداد نابه‌جایی‌های (inversion) آرایه‌ی اولیه است؛ یعنی تعداد زوج‌های (i, j) که i < j ولی A[i] > A[j]) دلیلش این است که هر جابه‌جایی فقط دو عنصر مجاور را عوض می‌کند و دقیقاً یک نابه‌جایی را از بین می‌برد، و آرایه‌ی نهایی صفر نابه‌جایی دارد. پس این عدد مستقل از جزئیات پیمایش است.

رابطه‌ی دوم: تعداد دورها برابر است با یک به‌علاوه‌ی بیشترین مسافتی که یک عنصر باید به سمت راست حرکت کند.

توضیح این رابطه به جهت پیمایش برمی‌گردد و کمی خلاف شهود اولیه است. چون در هر دور از انتها به سمت ابتدا حرکت می‌کنیم، وقتی به یک عنصر کوچک می‌رسیم آن را با همسایه‌ی چپش عوض می‌کنیم و بعد در گام بعدی دوباره همان عنصر را با همسایه‌ی جدیدش مقایسه می‌کنیم؛ یعنی عنصر همراه ما به سمت چپ «کشیده» می‌شود و می‌تواند در یک دور مسافت زیادی به چپ طی کند. در مقابل، عنصری که باید به راست برود در هر دور فقط یک خانه جابه‌جا می‌شود، چون بعد از عوض شدن، پیمایش از آن عبور کرده و به سمت چپ رفته است.

پس تعداد دورهای لازم برابر بیشینه‌ی (اندیس نهایی منهای اندیس اولیه) روی همه‌ی عناصر است و یک دور هم برای تایید نهایی اضافه می‌شود.

با این دو رابطه می‌توانید بدون اجرای شبیه‌سازی، درستی جواب دانشجو را روی هر ورودی بررسی کنید.

چند حالت خاص که تست‌ها را رد می‌کنند

- آرایه‌ی از پیش مرتب: در دور اول هیچ جابه‌جایی رخ نمی‌دهد و فرایند بعد از ۱ دور تمام می‌شود. خروجی: صفر جابه‌جایی و یک دور.
- آرایه‌ی تک‌عضوی: حلقه‌ی درونی range(0, 0, -1) است و اصلاً اجرا نمی‌شود، ولی یک دور کامل انجام شده. خروجی: صفر جابه‌جایی و یک دور. کسی که حلقه‌ی بیرونی را به‌صورت for i in range(n-1) نوشته باشد، اینجا صفر دور گزارش می‌کند و تست را رد می‌کند.
- آرایه‌ای که همه‌ی عناصرش برابرند: چون شرط جابه‌جایی «اکیداً کوچکتر» است، هیچ جابه‌جایی رخ نمی‌دهد. اگر کسی از => استفاده کرده باشد، در حلقه‌ی بی‌نهایت می‌افتد.
- آرایه‌ی کاملاً معکوس: بیشینه‌ی جابه‌جایی یعنی n(n-1)/2 و تعداد دورها برابر n است (n-1 دور برای مرتب شدن و یک دور تایید).
- آرایه‌ای که فقط کوچکترین عنصرش در انتهاست، مثل [n, 1, 2, 3, ...]: با اینکه n-1 جابه‌جایی لازم دارد، فقط ۲ دور طول

می‌کشد (تست ۷ دقیقاً حالت آینه‌ای همین است). چون عدد ۱ در همان دور اول تا ابتدای آرایه کشیده می‌شود و بقیه‌ی عناصر فقط یک خانه به راست رفته‌اند.

- آرایه‌ای که فقط بزرگترین عنصرش در ابتداست، مثل n, 1, 2, ..., [n-1]: این هم n-1 جابه‌جایی دارد ولی n دور طول می‌کشد، چون عدد n در هر دور فقط یک خانه به راست می‌رود. مقایسه‌ی این دو حالت نشان می‌دهد تعداد دورها هیچ تناسب ساده‌ای با تعداد جابه‌جایی‌ها ندارد، و همین دو ورودی کسانی را که چنین فرضی کرده‌اند رد می‌کند.

اشتباهات رایج و پاسخ به پرسش‌های احتمالی دانشجو

- زیاد کردن شمارنده‌ی دور در انتهای حلقه یا بعد از بررسی پرچم، که دور آخر را نمی‌شمارد.
- استفاده از حلقه‌ی درونی که در هر دور کوتاه‌تر می‌شود.

جواب نهایی مرتب‌سازی و تعداد جابه‌جایی‌ها تغییر نمی‌کند، ولی تعداد دورها می‌تواند متفاوت شود. صورت سوال صراحتاً پیمایش کامل آرایه در هر دور را خواسته است.

- پیمایش از ابتدا به انتها به‌جای انتها به ابتدا. این هم تعداد دورها را عوض می‌کند، چون در آن حالت عناصر بزرگ سریع به راست می‌روند ولی عناصر کوچک کند به چپ می‌آیند.
- استفاده از شرط <= در مقایسه که با عناصر تکراری به حلقه‌ی بی‌نهایت منجر می‌شود. اگر دانشجو گفت «برنامه‌ام روی یک تست تمام نمی‌شود»، اول همین را چک کنید.

پیچیدگی

- زمان: O(n^2) در بدترین حالت
- حافظه: O(n)

پاسخ سوال ۴ (۷۰ امتیاز)

توضیحات

مرتب‌سازی مبنایی تعمیم طبیعی مرتب‌سازی سطلی است. ایده این است: به‌جای اینکه سعی کنیم اعداد را یکباره و بر اساس کل مقدارشان مرتب کنیم، آن‌ها را چند بار و هر بار فقط بر اساس یک رقم مرتب می‌کنیم.

نسخه‌ی LSDکه در این سوال خواسته شده، از کم‌ارزش‌ترین رقم (یکان) شروع می‌کند و به سمت پرارزش‌ترین رقم می‌رود. هر دور شامل دو مرحله است:

• پخش کردن (Distribution) : آرایه را از چپ به راست پیمایش می‌کنیم و هر عدد را بر اساس رقم فعلی‌اش به انتهای یکی از ۱۰ سطل اضافه می‌کنیم. رقم فعلی عدد xبا فرمول (x // radix % 10)به دست می‌آید که در آن radixدر دور اول ۱، در دور دوم ۱۰، در دور سوم ۱۰۰ و ... است.
• جمع‌آوری(Collection): سطل‌ها را به ترتیب از ۰ تا ۹ پشت سر هم می‌چینیم و آرایه‌ی جدید را می‌سازیم.

چرا این الگوریتم درست کار می‌کند؟ (نقش حیاتی پایداری)

این مهم‌ترین بخش سوال است. ادعا می‌کنیم بعد از دور kام، آرایه بر اساس kرقم کم‌ارزش اعداد مرتب شده است. اثبات با استقرا:

پایه: بعد از دور اول، آرایه بر اساس رقم یکان مرتب است؛ واضح است.

گام استقرا: فرض کنید بعد از دور k-1ام، آرایه بر اساس k-1 رقم کم‌ارزش مرتب باشد. حالا دور kام را روی همین آرایه اجرا می‌کنیم. دو عدد دلخواه aو bرا در نظر بگیرید:

• اگر رقم kام آن‌ها متفاوت باشد، در سطل‌های متفاوتی می‌افتند و چون سطل‌ها به ترتیب جمع می‌شوند، ترتیبشان بر اساس رقم kام درست خواهد بود.
• اگر رقم kام آن‌ها یکسان باشد، هر دو در یک سطل می‌افتند. چون توزیع ما پایدار است (از چپ به راست می‌خوانیم و به انتهای سطل اضافه می‌کنیم)، ترتیب نسبی‌شان همان ترتیب دور قبلی می‌ماند که طبق فرض استقرا بر اساس k-1رقم کم‌ارزش درست بود.

پس در هر دو حالت ترتیب درست است و آرایه بعد از دور kام بر اساس kرقم کم‌ارزش مرتب می‌شود. بعد از dدور (dبرابر تعداد ارقام بزرگ‌ترین عدد)، آرایه کاملاً مرتب است.

اگر توزیع پایدار نباشد، این استدلال کاملاً فرو می‌ریزد و الگوریتم جواب غلط می‌دهد. به همین دلیل است که در جزوه تاکید شده در هر دور باید حتماً از یک مرتب‌سازی پایدار (سطلی یا شمارشی) استفاده کرد، نه یک مرتب‌سازی مقایسه‌ای دلخواه.

تحلیل پیچیدگی

در هر دور، پخش کردن O(n) و جمع‌آوری O(n + 10) طول می‌کشد. با d دور، پیچیدگی کل O(d × n) خواهد بود.

جالب است بدانید این الگوریتم از کران پایین Omega(n log n) مرتب‌سازی‌های مقایسه‌ای عبور می‌کند، چون اصلاً مقایسه‌ای انجام نمی‌دهد. البته این رایگان نیست: فرض کرده‌ایم اعداد صحیح نامنفی با تعداد ارقام محدود هستند. اگر d خیلی بزرگ شود (مثلاً رشته‌های با طول ۱۰۰۰)، ممکن است از O(n log n) هم بدتر شود.

چرا تست‌ها سخت‌اند

- تست ۲ آرایه‌ای است که همه‌ی عناصرش صفر است. حلقه‌ی while radix <= max_num در این حالت هیچ‌وقت اجرا نمی‌شود و برنامه چیزی چاپ نمی‌کند، در حالی که طبق صورت سوال باید دقیقاً یک سطر چاپ شود. این شایع‌ترین تست رد شده است.
- تست ۳ یک عدد پنج‌رقمی تنهاست و بررسی می‌کند که تعداد دورها درست حساب شده باشد.
- تست ۵ اعدادی با تعداد ارقام کاملاً متفاوت (از یک‌رقمی تا پنج‌رقمی) دارد.
- تست ۷ اعداد تکراری فراوان دارد؛ اگر پایداری را رعایت نکنید، ترتیب نهایی درست درمی‌آید ولی سطرهای میانی غلط می‌شوند.
- تست ۸ اعدادی دارد که فقط در پرارزش‌ترین رقم فرق می‌کنند، پس تا دور آخر هیچ اتفاقی نمی‌افتد.
- تست ۹ اعدادی دارد که ارقام کم‌ارزششان یکسان است، پس دورهای اول آرایه را دست‌نخورده می‌گذارند و همه‌ی کار در دورهای پایانی انجام می‌شود.

اشتباهات رایج و پاسخ به پرسش‌های احتمالی دانشجو

- محاسبه‌ی اشتباه تعداد دورها. استفاده از len(str(max(A))) اشکالی ندارد، ولی اگر تعداد دورها را از روی تعداد ارقام اولین عنصر یا تعداد عناصر حساب کنید جواب غلط می‌شود.

- فراموش کردن حالت خاصی که بزرگترین عدد صفر است. اگر دانشجو گفت «فقط یک تست را رد می‌شوم و خروجی‌ام خالی است»، دقیقاً همین است.
- به‌هم‌زدن پایداری، مثلاً با مرتب کردن محتوای هر سطل یا با پیمایش آرایه از راست به چپ.
- ثبت نکردن یک کپی از آرایه در گزارش، که باعث می‌شود همه‌ی سطرها یکسان چاپ شوند.
- استفاده از رقم پرارزش به‌جای کم‌ارزش برای شروع نسخه‌ی (MSD)آن الگوریتم هم درست است ولی خروجی سطرهای میانی‌اش با چیزی که صورت سوال خواسته فرق می‌کند.

پیچیدگی

- زمان: $O(d \times n)$
- حافظه: $O(n)$

پاسخ سوال ۵ (۱۰۰ امتیاز)

توضیحات

این سوال سه بخش دارد که هر کدام امتیاز خودش را دارد: پیاده‌سازی درست یک مرتب‌سازی پایدار، درک مرتب‌سازی چندکلیدی، و مدیریت دقیق ورودی‌های نامعتبر.

بخش اول: چرا مرتب‌سازی درجی پایدار است؟

مرتب‌سازی درجی را برای این انتخاب کردیم که پایداری در آن تقریباً رایگان به دست می‌آید. در حلقه‌ی داخلی، عنصر جاری (item)فقط تا وقتی به عقب شیفت داده می‌شود که عنصر قبلی اکیداً از آن بزرگتر باشد:

```
while i > 0 and a[i-1][pos] > item[pos]:
```

اگر به‌جای > از >= استفاده کنید، عنصر جاری از روی عنصر هم‌کلید خودش هم رد می‌شود و پایداری از بین می‌رود. این یکی از ظریف‌ترین باگ‌های ممکن در این سوال است، چون خروجی همچنان «مرتب» به نظر می‌رسد و فقط در تست‌های حاوی کلیدهای تکراری خودش را نشان می‌دهد.

برای حالت نزولی، فقط جهت مقایسه را برعکس می‌کنیم:

```
while i > 0 and a[i-1][pos] < item[pos]:
```

باز هم شرط اکید است، پس عناصر هم‌کلید جابه‌جا نمی‌شوند و ترتیب فعلی‌شان حفظ می‌گردد. این دقیقاً همان چیزی است که صورت سوال از نزولی پایدار می‌خواهد.

بخش دوم: مرتب‌سازی چندکلیدی، همان ایده‌ی مرتب‌سازی مبنایی
اگر بخواهیم لیستی را بر اساس چند کلید مرتب کنیم (مثلاً کلید اصلی نمره و کلید فرعی سن)، کافی است با یک مرتب‌سازی پایدار، ابتدا بر اساس کلید کم‌اهمیت‌تر و در پایان بر اساس کلید مهم‌تر مرتب کنیم.

چرا کار می‌کند؟ چون مرتب‌سازی آخر (بر اساس نمره) عناصری را که نمره‌ی متفاوت دارند به ترتیب درست می‌چیند، و عناصری که نمره‌ی یکسان دارند را دست نمی‌زند؛ یعنی همان ترتیبی که از مرحله‌ی قبل (سن) داشتند باقی می‌ماند.

این دقیقاً همان استدلالی است که در جزوه برای درستی مرتب‌سازی مبنایی آمده است: آنجا کلیدها رقم‌های یک عدد بودند و از رقم یکان به سمت رقم پرارزش پیش می‌رفتیم. اینجا کلیدها فیلدهای یک رکورد هستند و از فیلد کم‌اهمیت به سمت فیلد مهم پیش می‌رویم. توجه کنید که ترتیب برعکس (اول نمره و بعد سن) نتیجه‌ی کاملاً متفاوتی می‌دهد.

بخش سوم: مدیریت ورودی نامعتبر
بهترین کار این است که اعتبارسنجی را از منطق اصلی جدا کنید. در راه‌حل ارائه‌شده دو تابع کمکی داریم:
- تابع valid_name که طول و کاراکترهای مجاز نام را بررسی می‌کند.
- تابع parse_int که هم تبدیل به عدد صحیح را انجام می‌دهد و هم بازه را چک می‌کند، و در صورت خطا مقدار Noneبرمی‌گرداند.

با این ساختار، بدنه‌ی هر دستور فقط دو کار می‌کند: اول تعداد پارامترها را چک می‌کند و بعد اعتبار مقادیر را. هر شکستی باعث چاپ invalid input و رفتن به سطر بعدی می‌شود.

نکته‌ی مهم: تشخیص «نام تکراری» یا «رکورد پیدا نشد» ورودی نامعتبر محسوب نمی‌شود؛ این‌ها وضعیت‌های منطقی برنامه هستند و پیام مخصوص خودشان را دارند. خلط کردن این دو دسته یکی از رایج‌ترین دلایل رد شدن تست‌ها است.

چرا تست‌ها سخت‌اند
- تست ۲ فقط ورودی‌های نامعتبر دارد: کمبود و زیادی پارامتر، نام با کاراکتر غیرمجاز، نمره و سن خارج از بازه، فیلد ناشناخته، و exitبا پارامتر اضافه.

- تست ۳ لیست خالی، حذف رکورد ناموجود و افزودن نام تکراری را می‌آزماید.
- تست ۴ نام‌هایی با حروف بزرگ و کوچک دارد و ترتیب اسکی را می‌سنجد؛ ضمناً همه‌ی نمره‌ها برابرند تا پایداری sort score دیده شود.
- تست ۵ زنجیره‌ی کامل مرتب‌سازی چندکلیدی است.
- تست ۶ روی داده‌ای کار می‌کند که همه‌ی کلیدهایش تکراری است؛ کسی که از >= استفاده کرده باشد اینجا رد می‌شود.
- تست ۷ چند بار rsort پشت سر هم دارد تا پایداری در حالت نزولی بررسی شود.
- تست ۸ مقادیر مرزی نمره (۰ و ۱۰۰) و سن (۱ و ۱۲۰) را دارد؛ کسی که بازه را با < به‌جای <= چک کرده باشد این‌ها را نامعتبر می‌گیرد.
- تست ۹ حذف در میانه‌ی یک لیست مرتب‌شده را می‌آزماید تا مطمئن شویم ترتیب بقیه به هم نمی‌ریزد.
- تست ۱۰ همه‌ی دستورها را در یک دنباله‌ی طولانی ترکیب می‌کند.

تحلیل پیچیدگی

- دستور addو remove به‌خاطر جست‌وجوی خطی برای نام، O(n) هستند.
- دستورهای sortو rsort با مرتب‌سازی درجی در بدترین حالت O(n^2) هستند.

اگر بخواهید بهینه‌تر کار کنید، می‌توانید از یک دیکشنری برای نگاشت نام به رکورد استفاده کنید تا addو removeبه O(1) برسند، و برای مرتب‌سازی از Merge Sort پایدار با O(n log n) استفاده کنید.

اشتباهات رایج و پاسخ به پرسش‌های احتمالی دانشجو

- استفاده از >= به‌جای < در شرط حلقه‌ی داخلی که پایداری را از بین می‌برد. اگر دانشجو گفت «فقط تست‌های تکراری‌دار را رد می‌شوم»، دقیقاً همین است.
- پیاده‌سازی rsort با معکوس کردن خروجی .sort این کار ترتیب عناصر هم‌کلید را هم برعکس می‌کند و جواب غلط می‌دهد.
- استفاده از sortedیا .list.sort جالب است بدانید مرتب‌سازی پایتون (Timsort)واقعاً پایدار است و حتی sorted(a, reverse=True)

هم پایداری را حفظ می‌کند، ولی هدف آموزشی این سوال پیاده‌سازی دستی الگوریتم است.

- فراموش کردن حالت سطر خالی در ورودی.
- چاپ کردن :Recordsحتی وقتی لیست خالی است.
- بررسی نکردن پارامتر اضافه برای دستورهای بدون پارامتر مثل showو exit.